

Gioush 大队 2026 夏季常规赛

第二试题解

出题人：Ehundategh & tfbz

题号	题目	文件名
1	归零	<code>reset</code>
2	引力之阱	<code>escape</code>
3	商路照影	<code>trade</code>
4	逐风	<code>wind</code>

目录

归零 (reset)	2
题目简述	2
第一档部分分	2
特殊性质	2
正解	2
复杂度	2
引力之阱 (escape)	3
题目简述	3
32 分	3
64 分	3
正解	4
复杂度	4
商路照影 (trade)	5
题意简述	5
数据点 1-8	5
数据点 9-12	5
正解 1: 树状数组	5
正解 2: 线段树合并	6
代码	6
逐风 (wind)	8
题目简述	8
数据点 1-8	8
数据点 9-12	8
正解	9
代码	10

归零 (reset)

题目简述

给定 n 盏星灯，单独归零第 i 盏的代价为 a_i ，同时归零相邻的第 $i, i+1$ 盏的代价为 b_i 。每盏星灯必须恰好被归零一次，求最小总代价。

第一档部分分

这部分 n 很小，可以考虑直接搜索。

从左到右考虑第一盏还没有归零的星灯 i 。我们可以选择单独归零第 i 盏，也可以在 $i < n$ 时同时归零第 $i, i+1$ 盏。递归枚举所有选择即可。

特殊性质

若对于任意 $1 \leq i < n$ ，都有 $b_i = a_i + a_{i+1}$ ，那么同时归零相邻两盏星灯并不会比单独归零它们更优。

于是答案就是

$$\sum_{i=1}^n a_i.$$

正解

考虑最后一次操作。

若第 i 盏星灯是单独归零的，那么前 $i-1$ 盏星灯已经全部归零，代价为 $dp_{i-1} + a_i$ 。

若第 i 盏星灯和第 $i-1$ 盏星灯一起归零，那么前 $i-2$ 盏星灯已经全部归零，代价为 $dp_{i-2} + b_{i-1}$ 。

因此，令 dp_i 表示归零前 i 盏星灯的最小代价，可以得到：

$$dp_i = \min(dp_{i-1} + a_i, dp_{i-2} + b_{i-1}).$$

初始时 $dp_0 = 0, dp_1 = a_1$ 。从左到右递推即可。

复杂度

时间复杂度为 $\mathcal{O}(n)$ ，空间复杂度可以优化到 $\mathcal{O}(1)$ 。

引力之阱 (escape)

题目简述

给出 $n + 1$ 个 n 维点及其到某个未知点的距离平方，求这个未知点的坐标。

32 分

这部分 $n \leq 3$ ，且保证答案坐标均为整数，满足 $-30 \leq x_i \leq 30$ 。

根据题目描述，对于每个探测器 i ，合法的答案都需要满足

$$\sum_{j=1}^n (x_j - a_{i,j})^2 = d_i.$$

因此，可以直接枚举每一维坐标 x_i ，再检查枚举出的点是否满足全部 $n + 1$ 个等式。枚举的状态数不超过 61^3 ，可以通过这部分数据。

64 分

这部分满足特殊性质。设第 $i + 1$ 个探测器的第 i 维坐标为 c_i 。

由于第 1 个探测器位于原点，有

$$d_1 = \sum_{j=1}^n x_j^2.$$

第 $i + 1$ 个探测器只有第 i 维坐标不为 0，因此

$$d_{i+1} = (x_i - c_i)^2 + \sum_{j \neq i} x_j^2.$$

将两个式子相减，可以得到

$$d_{i+1} - d_1 = c_i^2 - 2c_i x_i,$$

于是

$$x_i = \frac{d_1 + c_i^2 - d_{i+1}}{2c_i}.$$

对每一维分别计算即可。更重要的是，这部分启发我们：将两个距离平方的等式相减，可以消去所有未知数的平方项。

正解

回到第一档部分分中的等式。对于任意 $2 \leq i \leq n+1$ ，将第 i 个探测器对应的等式减去第 1 个探测器对应的等式：

$$\sum_{j=1}^n (x_j - a_{i,j})^2 - \sum_{j=1}^n (x_j - a_{1,j})^2 = d_i - d_1.$$

根据上一档部分分的观察，展开后所有 x_j^2 都会被消去。整理得到

$$2 \sum_{j=1}^n (a_{i,j} - a_{1,j}) x_j = d_1 - d_i + \sum_{j=1}^n (a_{i,j}^2 - a_{1,j}^2).$$

对于每个 $2 \leq i \leq n+1$ 建立一个方程，就得到了一个包含 n 个未知数和 n 个方程的线性方程组，使用高斯消元求解即可。

题目保证答案唯一，因此高斯消元过程中一定能够找到非零主元。使用 `long double` 完成计算，并在输出时保留三位小数即可。

为了避免浮点误差导致输出 `-0.000`，若某一维答案的绝对值小于 `0.0005`，应当先将其赋值为 `0`。

复杂度

建立方程组的时间复杂度为 $\mathcal{O}(n^2)$ ，高斯消元的时间复杂度为 $\mathcal{O}(n^3)$ ，空间复杂度为 $\mathcal{O}(n^2)$ 。

商路照影 (trade)

题意简述

给定一棵有根二叉树，第 i 个节点的权值为 w_i 。对于每个节点 x ，统计满足 u 在 x 的左儿子子树内， v 在 x 的右儿子子树内，并且 $w_u < w_x < w_v$ 的有序点对 (u, v) 数量。

显然，每一个合法点对的中转点 x 都是唯一的。若记左儿子子树内权值小于 w_x 的节点数量为 L_x ，右儿子子树内权值大于 w_x 的节点数量为 R_x ，那么节点 x 对答案的贡献就是 $L_x R_x$ 。

数据点 1 ~ 8

这部分 $n \leq 2000$ 。

对于每个节点 x ，分别遍历其左儿子子树与右儿子子树，统计权值小于和大于 w_x 的节点数量即可，时间复杂度为 $\mathcal{O}(n^2)$ 。

数据点 9 ~ 12

特殊性质保证二叉树按照编号自然连接，因此整棵树的高度为 $\mathcal{O}(\log n)$ 。

考虑对于每个节点维护其子树内所有权值组成的有序序列。得到两个儿子的序列以后，可以通过二分查询出 L_x, R_x ，再使用归并排序的方法合并两个儿子的序列，并插入 w_x 。

每个权值在树的每一层至多参与一次归并，总时间复杂度为 $\mathcal{O}(n \log n)$ 。

正解 1：树状数组

首先对原树进行一次 DFS，求出每个节点的 DFN 序与子树大小。这样任意一个节点 u 的子树都对应 DFN 序上的连续区间

$$[\text{Dfn}(u), \text{Dfn}(u) + \text{Size}(u) - 1].$$

于是，求左儿子子树中权值小于 w_x 的节点数量，可以转化为一个二维偏序问题：节点的 DFN 序需要落在一段区间中，权值需要小于给定值。

将所有节点按照权值从小到大排序，并使用树状数组维护已经加入节点的 DFN 序。处理节点 x 时，树状数组中只保留权值小于 w_x 的节点，那么在左儿子子树对应的 DFN 区间上查询区间和，得到的就是 L_x 。

由于题目要求严格小于，所以相同权值的节点必须放在同一组中。对于一组权值相同的节点，应该先完成这一组的所有查询，再将这一组节点的 DFN 序加入树状数组。

同理，将所有节点按照权值从大到小处理，树状数组中只保留权值大于 w_x 的节点，在右儿子子树对应的区间上查询即可得到 R_x 。相同权值仍然需要先查询、后加入。

最后计算 $\sum_x L_x R_x$ 即可。排序的时间复杂度为 $\mathcal{O}(n \log n)$ ，树状数组操作的总时间复杂度为 $\mathcal{O}(n \log n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 2：线段树合并

先将所有权值离散化。对于每个树上节点 x ，维护一棵动态开点权值线段树 $Root_x$ ，其中记录 x 子树内每个离散化权值出现的次数。

递归处理节点 x 时，先处理左右儿子。此时，在线段树 $Root_{l_x}$ 中查询小于 w_x 的权值数量，得到 L_x ；在线段树 $Root_{r_x}$ 中查询大于 w_x 的权值数量，得到 R_x 。将 $L_x R_x$ 加入答案。

注意，必须先查询左右儿子的线段树，再进行合并。若先合并，就无法判断一个权值究竟来自左儿子子树还是右儿子子树。

查询完成以后，合并左右儿子的权值线段树，再将 w_x 插入合并后的线段树，得到 $Root_x$ 。

线段树合并时，若其中一个节点为空，直接返回另一个节点；否则递归合并左右儿子，并将区间计数相加。权值离散化的时间复杂度为 $\mathcal{O}(n \log n)$ ，线段树查询、插入与合并的总时间复杂度为 $\mathcal{O}(n \log n)$ ，空间复杂度为 $\mathcal{O}(n \log n)$ 。

代码

```
#include <cstdio>
#include <cstring>
#include <algorithm>
#define MAXN 200010
#define MAXM 4400010
using namespace std;
int n, LeftSon[MAXN], RightSon[MAXN], Val[MAXN], SortVal[MAXN], Rank[MAXN];
int Root[MAXN], LSon[MAXM], RSon[MAXM], Sum[MAXM], cnt=0;
long long Ans=0;
void Add(int &Now, int Left, int Right, int Pos){
    if(!Now) Now=++cnt;
    Sum[Now]++;
    if(Left==Right) return;
    int Mid=(Left+Right)>>1;
    if(Pos<=Mid) Add(LSon[Now], Left, Mid, Pos);
    else Add(RSon[Now], Mid+1, Right, Pos);
}
```

```

int Query(int Now,int Left,int Right,int L,int R){
    if(!Now||L>Right||R<Left) return 0;
    if(L<=Left&&Right<=R) return Sum[Now];
    int Mid=(Left+Right)>>1;
    return Query(LSon[Now],Left,Mid,L,R)+Query(RSon[Now],Mid+1,Right,L,R);
}

int Merge(int x,int y){
    if(!x||!y) return x+y;
    Sum[x]+=Sum[y];
    LSon[x]=Merge(LSon[x],LSon[y]);
    RSon[x]=Merge(RSon[x],RSon[y]);
    return x;
}

void DFS(int Now){
    if(LSon[Now]) DFS(LSon[Now]);
    if(RSon[Now]) DFS(RSon[Now]);
    int LeftCnt=0,RightCnt=0;
    if(LSon[Now]&&Rank[Now]>1) LeftCnt=Query(Root[LSon[Now]],1,n,1,Rank[Now]-1);
    if(RSon[Now]&&Rank[Now]<n) RightCnt=Query(Root[RSon[Now]],1,n,Rank[Now]+1,n);
    Ans+=1ll*LeftCnt*RightCnt;
    Root[Now]=Merge(Root[LSon[Now]],Root[RSon[Now]]);
    Add(Root[Now],1,n,Rank[Now]);
}

int main(){
    freopen("trade.in","r",stdin);
    freopen("trade.out","w",stdout);
    scanf("%d",&n);
    for(int i=1;i<=n;i++) scanf("%d",&Val[i]),SortVal[i]=Val[i];
    for(int i=1;i<=n;i++) scanf("%d%d",&LSon[i],&RSon[i]);
    sort(SortVal+1,SortVal+n+1);
    int Len=unique(SortVal+1,SortVal+n+1)-SortVal-1;
    for(int i=1;i<=n;i++) Rank[i]=lower_bound(SortVal+1,SortVal+Len+1,Val[i])-SortVal;
    DFS(1);
    printf("%lld\n",Ans);
    return 0;
}

```


逐风 (wind)

题目简述

给定周期为 n 的风向序列。每天白天可以自行移动不超过 k 的曼哈顿距离，傍晚会被当天的风推动。对于每组数据，要求最早在某天傍晚恰好到达目标点。

数据点 1 ~ 8

这部分保证答案存在且不超过 10^5 。

考虑枚举经过的天数 t 。设前 t 天的风一共使船只移动到 $(W_x(t), W_y(t))$ ，那么白天需要补充的位移为

$$(x - W_x(t), y - W_y(t)).$$

经过 t 天，白天最多可以移动 $t \times k$ 的曼哈顿距离。因此， t 天能够到达终点，当且仅当

$$|x - W_x(t)| + |y - W_y(t)| \leq t \times k.$$

从 $t = 0$ 开始依次枚举，并直接模拟每天的风即可。由于答案不超过 10^5 ，时间复杂度为 $\mathcal{O}(n + \text{Ans})$ 。

数据点 9 ~ 12

这部分满足特殊性质，即每天风的曼哈顿距离都不超过 k 。

沿用上一档部分的判定条件。假设经过 t 天可以到达终点，那么

$$|x - W_x(t)| + |y - W_y(t)| \leq t \times k.$$

从第 t 天到第 $t + 1$ 天，风带来的新增位移的曼哈顿距离不超过 k 。根据三角不等式，有

$$\begin{aligned} & |x - W_x(t + 1)| + |y - W_y(t + 1)| \\ & \leq |x - W_x(t)| + |y - W_y(t)| + k \\ & \leq (t + 1)k. \end{aligned}$$

所以，如果 t 天可行，那么 $t + 1$ 天也可行，判定具有单调性。预处理一个周期内的风位移前缀和后，可以二分最早的可行天数。

时间复杂度为 $\mathcal{O}(n + \log V)$ ，其中 V 是二分值域。

正解

一般情况下，单日风的位移可能超过 k ，上一档部分分中的单调性不再成立，不能直接二分答案。

设一个完整周期的风位移为 (S_x, S_y) ，前 t 天的风位移为 $(P_x(t), P_y(t))$ ，其中 $1 \leq t \leq n$ 。考虑答案除以 n 后的余数对应 t ，并设在这之前已经经过了 q 个完整周期，那么经过的总天数为

$$qn + t,$$

风带来的总位移为

$$(qS_x + P_x(t), qS_y + P_y(t)).$$

令

$$a = x - P_x(t), \quad b = y - P_y(t),$$

那么需要满足

$$|a - qS_x| + |b - qS_y| \leq (qn + t)k.$$

利用

$$|X| + |Y| = \max(X + Y, -X + Y, X - Y, -X - Y),$$

可以将绝对值限制拆成下面四个关于 q 的一次不等式：

$$\begin{cases} (nk + S_x + S_y)q \geq a + b - tk, \\ (nk - S_x + S_y)q \geq -a + b - tk, \\ (nk + S_x - S_y)q \geq a - b - tk, \\ (nk - S_x - S_y)q \geq -a - b - tk. \end{cases}$$

对于一个形如 $Aq \geq B$ 的不等式，并限制 q 为非负整数：

- 若 $A > 0$ ，可以得到 q 的下界；
- 若 $A = 0$ ，只需要判断该不等式是否恒成立；
- 若 $A < 0$ ，可以得到 q 的上界。

将四个不等式得到的上下界取交，若交集非空，取其中最小的 q ，即可得到当前 t 对应的最早时间。枚举 $1 \leq t \leq n$ ，取所有合法时间的最小值即为答案。

特别地， $t = n$ 可以表示经过整数个完整周期的情况；而时间为 0 的情况需要单独判断目标点是否为原点。

每个 t 只需要处理四个一次不等式，因此每组数据的时间复杂度为 $\mathcal{O}(n)$ ，总时间复杂度为 $\mathcal{O}(\sum n)$ 。

代码

```
#include <cstdio>
#include <cstring>
#include <algorithm>
using namespace std;
const long long INF=1e18;
int T;
long long n,k,x,y,X[100010],Y[100010];
long long PreX,PreY,Ans=0,NowX,NowY;
void Deal(long long a,long long b,long long &Max,long long &Min){
    if (a==0&&b>0) {Max=0,Min=INF;}
    if (a>0&&b>0) {
        Min=max(Min,(a+b-1)/a);
    }
    if (a<0) {
        if (b>0) {Max=0,Min=INF;}
        else if (b==0) {Max=min(Max,0ll);Min=max(Min,0ll);}
        else Max=min(Max,b/a);
    }
    return;
}
long long Get(long long a,long long b,long long t) {
    long long Max=INF,Min=0;
    Deal(n*k+PreX+PreY,a+b-t*k,Max,Min);
    Deal(n*k-PreX+PreY,-a+b-t*k,Max,Min);
    Deal(n*k+PreX-PreY,a-b-t*k,Max,Min);
    Deal(n*k-PreX-PreY,-a-b-t*k,Max,Min);
    if (Max>=Min) return Min*n+t;
    else return INF;
}
void Solve() {
    PreX=PreY=NowX=NowY=0;Ans=INF;
    scanf("%lld%lld%lld%lld",&n,&k,&x,&y);
```

```
for (int i=1;i<=n;i++) {
    scanf("%lld%lld",&X[i],&Y[i]);
    PreX+=111*X[i];
    PreY+=111*Y[i];
}
if (x==0&&y==0) {puts("0");return;}
for (int i=1;i<=n;i++) {
    NowX+=111*X[i];
    NowY+=111*Y[i];
    Ans=min(Get(x-NowX,y-NowY,i),Ans);
}
if (Ans==INF) puts("-1");
else printf("%lld\n",Ans);
return;
}
int main() {
    scanf("%d",&T);
    while (T-->0) Solve();
    return 0;
}
```